

ALAB 351.2 — Data Types, Variables, Operators, and Basic I/O

Name: James Sloan

Date: August 18, 2026

Repository: <https://github.com/santigrey/Python-Essentials> — modules/Module 351/ALAB-351.2/

Files Submitted

File	Part	Purpose
types_and_vars.py	Part 1	Variables, data types, arithmetic and string operators
simple_calculator.py	Part 2	Two-number calculator with input validation
string_fun.py	Bonus	Word length, uppercase, and repetition
README.md	—	Run instructions and example output for each script

Part 1: Python Basics and Output — `types_and_vars.py`

Code

```
# ALAB 351.2 – Part 1: Data Types, Variables, and Operators
# James Sloan
#
# Purpose: declare variables of three different data types, use them in an
# introduction sentence, calculate a future age and a rectangle area, and
# demonstrate arithmetic and string operators.

# -----
# Section 1: Variable declarations
# Each variable holds a different data type:
#   str   – text, written in quotes
#   int   – a whole number, no decimal point
#   float – a number with a decimal point
# -----

name = "James"      # str
age = 41             # int
height = 1.83        # float, measured in meters

print("Section 1: Variables and their data types")
print("-----")

# type(x).__name__ prints the type as plain text ("str") instead of "<class 'str'>".
print(f"name   = {name!r}   -> {type(name).__name__}")
print(f"age    = {age}      -> {type(age).__name__}")
print(f"height = {height}   -> {type(height).__name__}")
print()

# -----
# Section 2: Introduction sentence
# An f-string drops each variable straight into the sentence inside {}.
# The :.2f formats the float to exactly two decimal places.
# -----

print("Section 2: Introduction")
print("-----")
print(f"Hello, my name is {name}. I am {age} years old and {height:.2f} meters tall.")
print()

# -----
```

```

# Section 3: Age in 5 years
# The + operator on two ints does ordinary addition.
# -----
print("Section 3: Age in five years")
print("-----")
years_ahead = 5
future_age = age + years_ahead          # addition
print(f"In {years_ahead} years, I will be {future_age} years old.")
print()

# -----
# Section 4: Area of a rectangle (width 5.5, height 2)
# The * operator multiplies. rect_width is a float and rect_height is an int,
# so Python promotes the result to a float automatically.
# The rectangle's measurements are named rect_width and rect_height so they do
# not overwrite the personal `height` variable declared in Section 1.
# -----
print("Section 4: Rectangle area")
print("-----")
rect_width = 5.5          # float
rect_height = 2           # int
area = rect_width * rect_height      # multiplication
print(f"The area of a {rect_width} x {rect_height} rectangle is {area}.")
print()

# -----
# Section 5: More operators
# Arithmetic operators demonstrated here:
#   + addition          - subtraction      * multiplication
#   / true division     // floor division  % remainder (modulo)
#   ** exponent
# String operators demonstrated here:
#   + concatenation (joins strings)
#   * repetition (repeats a string)
#   .upper() / .lower() / len() string methods and the built-in length function
# -----
print("Section 5: Operators")
print("-----")

# Arithmetic
print(f"age + 5      = {age + 5}")          # addition
print(f"age - 5      = {age - 5}")          # subtraction
print(f"age * 2       = {age * 2}")          # multiplication

```

```
print(f"age / 2      = {age / 2}")          # true division always gives a float
print(f"age // 2    = {age // 2}")          # floor division throws away the remainder
print(f"age % 2     = {age % 2}")          # remainder: 1 means the number is odd
print(f"height ** 2 = {height ** 2:.4f}")    # exponent

# Strings
first_name = "James"
last_name = "Sloan"
full_name = first_name + " " + last_name    # concatenation with +
print(f'"{first_name}" + " " + "{last_name}" = "{full_name}"')
print(f'{"-" * 20} = "{"-" * 20}''')        # repetition with *
print(f'"{full_name}".upper() = "{full_name.upper()}"')
print(f'len("{full_name}") = {len(full_name)}')
```

Output

This script takes no input, so it prints the same thing every run.

Section 1: Variables and their data types

```
name = 'James' -> str
age = 41 -> int
height = 1.83 -> float
```

Section 2: Introduction

Hello, my name is James. I am 41 years old and 1.83 meters tall.

Section 3: Age in five years

In 5 years, I will be 46 years old.

Section 4: Rectangle area

The area of a 5.5 x 2 rectangle is 11.0.

Section 5: Operators

```
age + 5 = 46
age - 5 = 36
age * 2 = 82
age / 2 = 20.5
age // 2 = 20
age % 2 = 1
height ** 2 = 3.3489
"James" + " " + "Sloan" = "James Sloan"
"_" * 20 = "_____"
"James Sloan".upper() = "JAMES SLOAN"
len("James Sloan") = 11
```

Requirements Coverage

Requirement	Where it is met
Variables for name (string), age (integer), height (float in meters)	Section 1 — <code>name</code> , <code>age</code> , <code>height</code> , each labeled with its type
Introduction sentence using those variables, not hardcoded values	Section 2 — one f-string with all three variables
Age in 5 years, with a printed statement	Section 3 — <code>age + years_ahead</code> , printed as <code>In 5 years, I will be 46 years old.</code>
Area of a rectangle with width 5.5 and height 2, formatted output	Section 4 — <code>rect_width * rect_height</code> , printed as <code>The area of a 5.5 x 2 rectangle is 11.0.</code>
At least two arithmetic operators and one string operation	Section 5 — seven arithmetic operators (<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>//</code> , <code>%</code> , <code>**</code>) and four string operations (<code>+</code> , <code>*</code> , <code>.upper()</code> , <code>len()</code>)
Comments explaining each section	Every section opens with a comment block; individual operators are annotated inline

Part 2: User Interaction and Input — `simple_calculator.py`

Code

```
# ALAB 351.2 – Part 2: Simple Calculator
# James Sloan
#
# Asks for two numbers and an operation, then prints the result.
# Bad input never crashes the program – it just asks again.

print("Simple Calculator")
print("=====")

# -----
# Step 1: Read the first number.
# input() always hands back a string, so float() converts it to a number.
# If the text is not a number, float() raises a ValueError. The try/except
# catches that, prints a message, and the while loop asks again.
# -----
while True:
    first_text = input("Enter the first number: ").strip()
    try:
        first = float(first_text)
        break
    except ValueError:
        print(f" '{first_text}' is not a number. Please try again.")

# -----
# Step 2: Read the second number the same way.
# -----
while True:
    second_text = input("Enter the second number: ").strip()
    try:
        second = float(second_text)
        break
    except ValueError:
        print(f" '{second_text}' is not a number. Please try again.")

# -----
# Step 3: Read the operation and check it against the four allowed symbols.
# -----
while True:
```

```

operation = input("Choose an operation (+, -, *, /): ").strip()
if operation in ("+", "-", "*", "/"):
    break
print(f" '{operation}' is not one of +, -, *, /. Please try again.")

# -----
# Step 4: Do the math with an if/elif/else chain.
# division_by_zero is a flag so the message below knows dividing failed.
# -----
division_by_zero = False

if operation == "+":
    result = first + second
elif operation == "-":
    result = first - second
elif operation == "*":
    result = first * second
else:
    # the only symbol left is "/"
    if second == 0:
        division_by_zero = True
        # dividing by zero is undefined
        result = None
    else:
        result = first / second

# -----
# Step 5: Print the result in a friendly "7 * 3 = 21" format.
# float() turns every input into a float, so 7 comes back as 7.0. Comparing a
# number to int(number) tells me whether it is really a whole number, and if it
# is I print the int version so the line reads 21 instead of 21.0.
# -----
first_display = int(first) if first == int(first) else first
second_display = int(second) if second == int(second) else second

print()
if division_by_zero:
    print(f"{first_display} / {second_display} = undefined")
    print("Error: division by zero is not allowed.")
else:
    # round() keeps long decimals such as 10 / 3 readable.
    result = round(result, 4)
    result_display = int(result) if result == int(result) else result
    print(f"{first_display} {operation} {second_display} = {result_display}")

```


Sample Runs

Multiplication — the exact format the lab asks for:

```
Simple Calculator
=====
Enter the first number: 7
Enter the second number: 3
Choose an operation (+, -, *, /): *

7 * 3 = 21
```

Addition with a negative number and a decimal:

```
Simple Calculator
=====
Enter the first number: -12
Enter the second number: 4.25
Choose an operation (+, -, *, /): +

-12 + 4.25 = -7.75
```

Subtraction:

```
Simple Calculator
=====
Enter the first number: 15.5
Enter the second number: 4
Choose an operation (+, -, *, /): -

15.5 - 4 = 11.5
```

Division producing a long decimal — rounded to four places:

```
Simple Calculator
=====
Enter the first number: 10
Enter the second number: 3
Choose an operation (+, -, *, /): /

10 / 3 = 3.3333
```

Invalid number, then an invalid operator — the program re-asks instead of crashing:

```
Simple Calculator
=====
Enter the first number: abc
    'abc' is not a number. Please try again.
Enter the first number: 15.5
Enter the second number: 4
Choose an operation (+, -, *, /): ^
    '^' is not one of +, -, *, /. Please try again.
Choose an operation (+, -, *, /): -

15.5 - 4 = 11.5
```

Division by zero:

```
Simple Calculator
=====
Enter the first number: 8
Enter the second number: 0
Choose an operation (+, -, *, /): /

8 / 0 = undefined
Error: division by zero is not allowed.
```

Requirements Coverage

Requirement	Where it is met
Two numeric inputs via separate <code>input()</code> calls, converted to a number	Steps 1 and 2 — two separate <code>input()</code> calls, each wrapped in <code>float()</code>
Prompt the user to select an operation (+, -, *, /)	Step 3
Perform the chosen calculation	Step 4
User-friendly result format, e.g. <code>7 * 3 = 21</code>	Step 5 — whole numbers print without a trailing <code>.0</code>
Handle invalid input gracefully with error messages	Non-numeric text, an unknown operator, and division by zero all produce a plain-English message; the first two re-prompt
Use if/elif/else structures	Step 4 is a four-branch <code>if / elif / elif / else</code> chain
Include comments	Each step opens with a comment block explaining what it does and why

Bonus: string_fun.py

Code

```
# ALAB 351.2 – Bonus: String Fun
# James Sloan
#
# Asks for a single word and reports a few facts about it.

print("String Fun")
print("=====")

# -----
# Ask for a word. .strip() removes any spaces typed before or after it, so
# " cat " is treated the same as "cat".
# -----
word = input("Enter a word: ").strip()

# If the user just pressed Enter there is nothing to work with, so use a
# default word instead of printing empty results.
if word == "":
    word = "Python"
    print("(No word entered – using 'Python' instead.)")

print()

# --- Required: length -----
# len() counts the characters in a string.
print(f"Length:      {len(word)} characters")

# --- Required: uppercase -----
# .upper() returns a new string in capitals; the original is unchanged.
print(f"Uppercase:    {word.upper()}")

# --- Required: repeated three times -----
# The * operator on a string repeats it. Adding a space first keeps the three
# copies readable instead of running them together.
print(f"Repeated x3:  {(word + ' ') * 3}.rstrip()")

# --- Extras -----
print(f"Lowercase:     {word.lower()}")
print(f"Title case:    {word.title()}")
```

```

print(f"First letter: {word[0]}")          # index 0 is the first character
print(f"Last letter: {word[-1]}")         # index -1 counts from the end
print(f"Reversed: {word[::-1]}")          # a slice with step -1 reverses it

# A palindrome reads the same forwards and backwards. Comparing the lowercase
# word to its reverse checks that in one line.
if word.lower() == word.lower()[::-1]:
    print(f"Palindrome:    yes, '{word}' reads the same backwards")
else:
    print("Palindrome:    no")

```

Sample Runs

```

String Fun
=====
Enter a word: Python

Length:      6 characters
Uppercase:   PYTHON
Repeated x3: Python Python Python
Lowercase:   python
Title case:  Python
First letter: P
Last letter: n
Reversed:    nohtyP
Palindrome:  no

```

```

String Fun
=====
Enter a word: racecar

Length:      7 characters
Uppercase:   RACECAR
Repeated x3: racecar racecar racecar
Lowercase:   racecar
Title case:  Racecar
First letter: r
Last letter: r
Reversed:    racecar
Palindrome:  yes, 'racecar' reads the same backwards

```

The three required lines are the length, the uppercase form, and the word repeated three times. The rest is extra: lowercase, title case, the first and last characters, the word reversed, and a palindrome check that reuses the reversed string.

Challenges Encountered and How I Resolved Them

Everything came back as a float. `float()` was the obvious way to convert the input, but it meant `7 * 3` printed as `21.0` rather than the `21` the lab asks for. Using `int()` instead would have rejected decimals, so I kept `float()` and fixed it at the printing step: comparing a number to `int(number)` tells me whether it is really a whole number, and if it is, I print the int version. That keeps decimal input working and still prints `7 * 3 = 21`.

Long division results looked bad. `10 / 3` printed seventeen digits. I wrapped the result in `round(result, 4)`, which keeps the line readable without changing whole-number answers.

Division by zero crashed the program. My first version raised `ZeroDivisionError` and stopped. Since the lab asks for graceful handling, I added an `if second == 0` check inside the division branch and set a flag so the printing step could show `8 / 0 = undefined` plus an explanation instead of a traceback.

A stray space broke the operator check. Typing `*` with a trailing space failed the comparison against `"*"`. Adding `.strip()` to each `input()` call fixed that, and it also handles spaces typed around a number.

What I Researched

Three things went beyond the lesson material.

try / except. `float("abc")` raises a `ValueError` and stops the program. Wrapping the conversion in `try` lets me catch that error and print a message instead of crashing. This is the standard Python pattern for "attempt a conversion, deal with it if it fails."

while True with break. Printing an error message is only half of handling bad input — the program still needs the value. Putting the prompt inside a `while True` loop and calling `break` only after a successful conversion means the user gets asked again until the input is valid.

Checking membership with in. Instead of writing `if op == "+" or op == "-" or op == "*" or op == "/"`, the expression `if operation in ("+", "-", "*", "/")` checks all four at once and is easier to read.

I also confirmed two smaller points: `/` always produces a float even when both operands are whole numbers (`4 / 2` is `2.0`), which is why `//` exists for whole-number division; and `word[::-1]` reverses a string using slice notation with a step of `-1`.